Laboratory of Advanced Programming

2026-04-01

Seismic Sentinel

Distributed, Fault-Tolerant Seismic Analysis System

Lorenzo Baudo
Flavio Della Penna
Alessio Kika

Contents

01 Mission & Goals

02 Architecture & Stack

03 Persistence & Data Model

04 Broker Fan-Out

05 Processing & Detection

06 Gateway & Frontend

Mission Briefing

Setting

2038 geopolitical crisis → seismic signals used for threat detection

Inputs

worldwide sensors streaming continuously

Constraint (Kicker)

Neutral zone can only route/forward—no intelligence processing

Requirement

keep detection running even during replica failures

Objective

- Real-time ingest of seismic samples via WebSocket
- Distributed frequency analysis (FFT) close to compute
- Classify events by dominant frequency band
- Persist duplicate-safe events
- Serve a live dashboard for filtering and monitoring

INGEST

ANALYZE

CLASSIFY

PERSIST

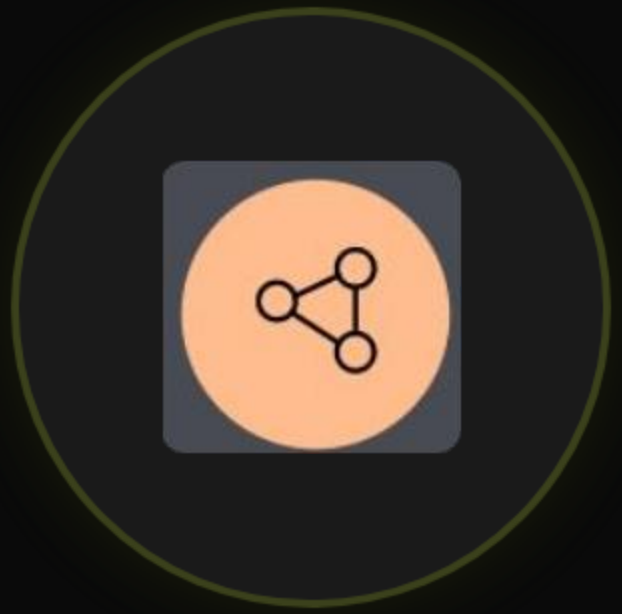
MONITOR

Data Flow

Processing replicas do compute + detection
Simulator
Broker
Processing Replicas
PostgreSQL
Gateway
Frontend
Broker forwards only (neutral zone compliant)
Gateway exposes read APIs to UI

Tech Stack

Layer	Technology	Notes
Backend API	Python + FastAPI	microservices
Async	asyncio + websockets	streaming + concurrency
Database	PostgreSQL	shared persistence
Frontend	React	dashboard UI
Container	Docker + Compose	reproducible run



PostgreSQL as Shared Source of Truth

All replicas write to the same database

Supports:

deduplication via unique keys

consistent reads for gateway/dashboard

Operational note:

schema optimized for fast filtering

DB Schema

detected_events

event_id	VARCHAR (PK)
sensor_id	VARCHAR
timestamp	TIMESTAMP
dominant_frequency_hz	FLOAT
peak_amplitude	FLOAT
Sensor_name	VARCHAR
Sensor_region	VARCHAR
event_type	VARCHAR
Window_start/end	TIMESTAMP
Detected_at	TIMESTAMP
Replica_id	VARCHAR

Docker Infrastucture

- Multi-Service Architecture :
  Each component runs in its own Docker Container
- Docker-Based Deployment :
  Every service has its own Dockerfile
- Orchestration :
  Docker-compose.yaml manages the entire system
- One-Step Startup :
  System fully reproducible with a single command :
  Docker compose up

Deployment & Reproducibility

Containers

Service	Image / Build	Host Port	Depends On
simulator	seismic-signal-simulator	8080	-
postgres	postgres:16	5432	-
processing-1/2/3	build ./processing-service	8101-8103	postgres, simulator
broker	build ./broker-service	8001	simulator, processing
gateway	build ./gateway-service	8000	postgres
frontend	build ./frontend-dashboard	3000	gateway

Custom Broker

Role

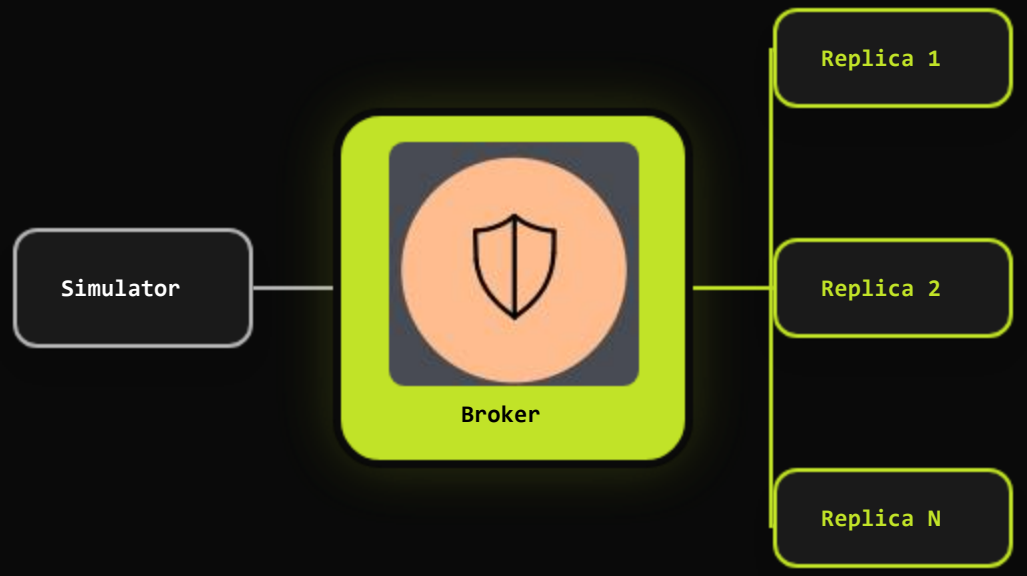
ingest from simulator + broadcast to all active replicas

Neutral-zone fit

routing/forwarding only

Benefits

- simple responsibilities
- minimal state → easy restart
- deterministic delivery to replicas



Processing Replicas

- BROADCAST MODEL :

  Each replica receives the same stream

- INDIPENDENT PROCESSING :

  Each replica processes independently (no shared in-memory state)

- SHARED PERSISTENCE

  Output: detected event records written to shared DB

System keeps working if some replicas fail

```
# 1. Add measurement to sliding window
window_manager.add_measurement(sensor_id, measurement)

# 2. Check if window is ready for analysis
if window_manager.is_window_ready(sensor_id) and \
   window_manager.should_analyze(sensor_id):

    window = window_manager.get_window(sensor_id)
    samples = [m["value"] for m in window]

    # 3. Frequency analysis (FFT)
    spectrum = analyze_frequency_spectrum(samples, SAMPLING_RATE_HZ)
    freq = spectrum["classifiable_dominant_frequency_hz"]

    # 4. Event classification
    event_type = classify_event(freq)
```

Sliding Window

- Window size: 5 s
- Sample rate: 20 Hz
- Samples/window: 100

Purpose: stable FFT input + continuous detection

FFT Analysis

- Transform: time-domain → frequency-domain (FFT/DFT)
- Extract:
  - dominant frequency
  - peak amplitude

Output is used for event classification

```python
def _compute_spectrum(signal: np.ndarray, sampling_rate_hz: float):
    window = np.hanning(len(signal))
    windowed = signal * window

    padded_length = max(len(signal) * 4, len(signal))
    fft_result = np.fft.rfft(windowed, n=padded_length)
    magnitudes = np.abs(fft_result)
    frequencies = np.fft.rfftfreq(padded_length, d=1.0 / sampling_rate_hz)

    if len(magnitudes) > 0:
        magnitudes[0] = 0.0

    return frequencies, magnitudes
```

Event Classification

01

Earthquake

0.5–3 Hz

02

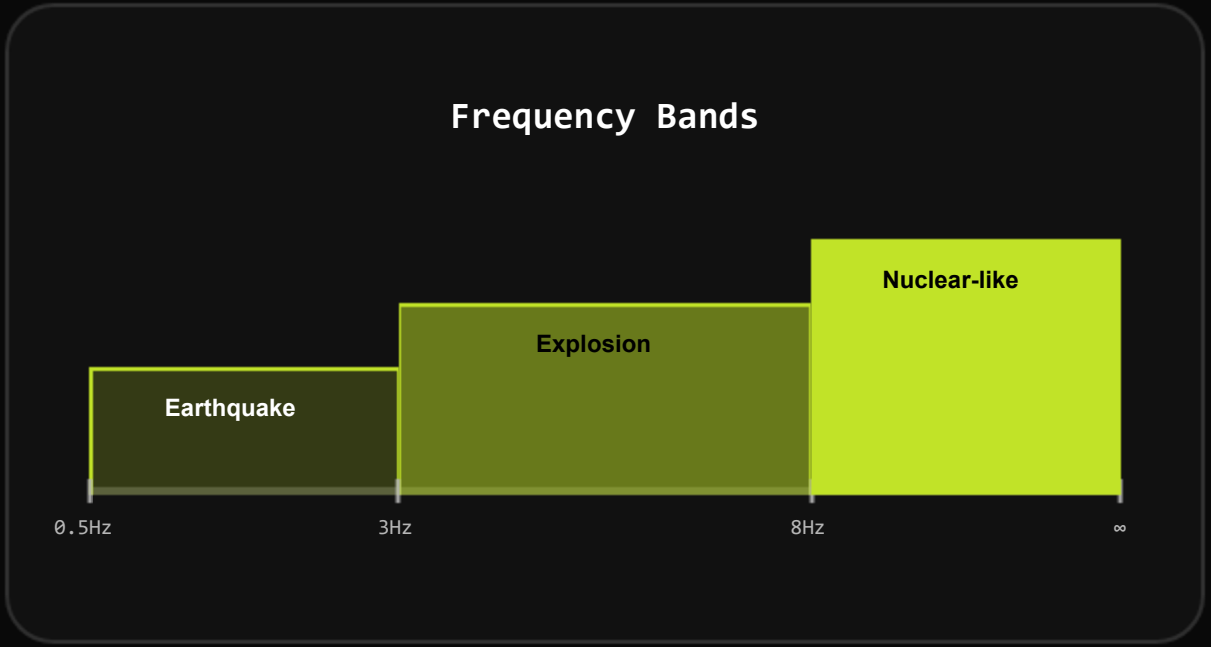
Explosion

3–8 Hz

03

Nuclear-like

≥8 Hz



Duplicate Handling

1. **Deterministic event_id**

   sensor_id + event_type + window + frequency

2. **DB conflict handling**

   ignore duplicates on insert

3. **Cooldown**

   prevent rapid re-inserts of same event

Compute event_id

Insert

On conflict do nothing

Gateway

Gateway

- single entry point
- filtering endpoints
- health/status aggregation

```python
@app.get("/system/status")
def get_system_status():
    replicas_health = check_replicas_health()
    system_status = compute_system_status(replicas_health)

    return {
        "status": system_status,
        "active_replicas": sum(1 for s in replicas_health.values() if s == "UP"),
        "total_replicas": len(replicas_health),
    }
```

Frontend Dashboard

Distributed Seismic Dashboard

Real-time monitoring of detected seismic events retrieved through the gateway service.

DASHBOARD STATUS:	EVENTS SHOWN IN PAGE:	CURRENT PAGE:	LAST UPDATE:
Loading...	25	2	31/03/2026, 18:02:28

System Status

System: DEGRADED

Active replicas: 2 / 3

- http://processing-1:8100 → **DOWN**
- http://processing-2:8100 → **UP**
- http://processing-3:8100 → **UP**

Frontend Dashboard

Sensor ID: e.g. sensor-12
Event Type: All
Sensor Region: e.g. north-zone
Rows per page: 25
Order: Newest first

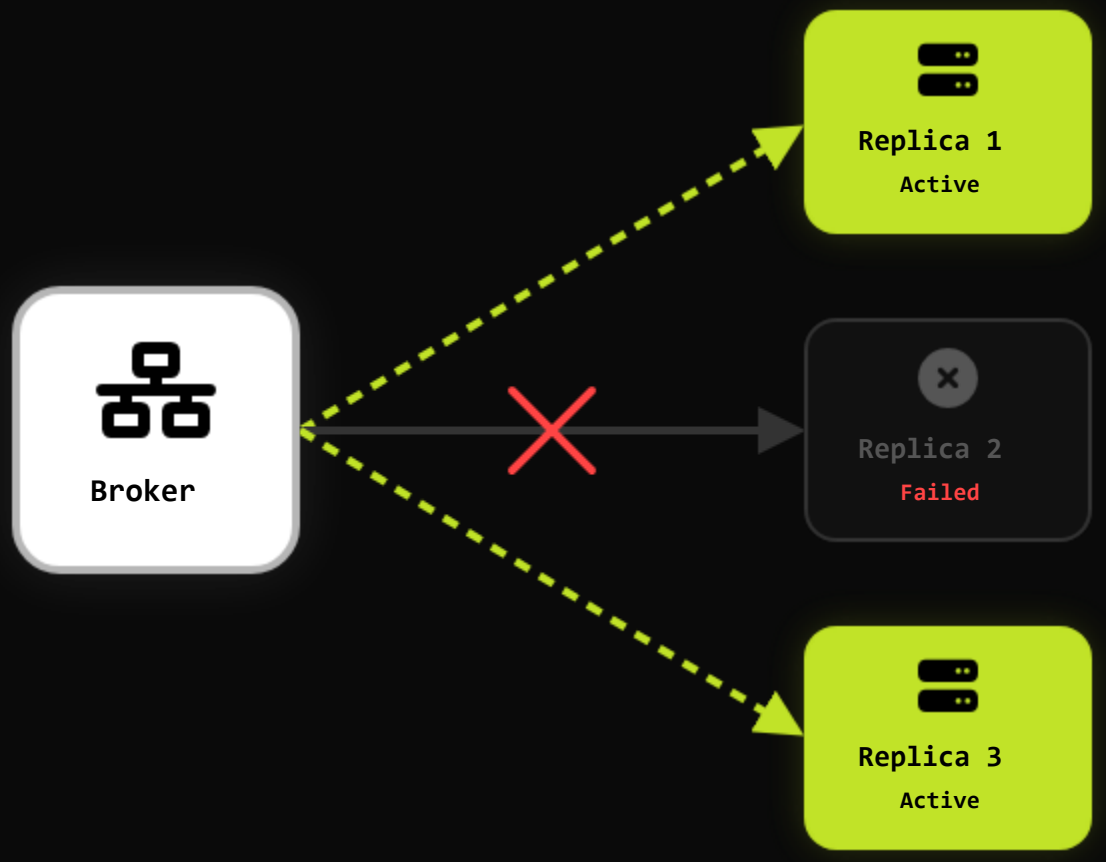
Refresh | Clear filters

Detected At	Sensor ID	Sensor Name	Region	Event Type	Frequency (Hz)	Peak Amplitude	Replica
31/03/2026, 18:02:09	sensor-05	Helios Shelf	Aegean Arc	earthquake	0.66	0.2545	processing-2
31/03/2026, 18:02:08	sensor-05	Helios Shelf	Aegean Arc	nuclear_like	9.63	0.3247	processing-2
31/03/2026, 18:02:04	sensor-09	DC East Cooling Hall	Replica Datacenter	conventional_explosion	6.09	4.9779	processing-3
31/03/2026, 18:02:01	sensor-11	DC West Access Tunnel	Replica Datacenter	earthquake	0.83	0.3006	processing-3
31/03/2026, 18:02:00	sensor-11	DC West Access Tunnel	Replica Datacenter	conventional_explosion	5.72	0.3006	processing-3
31/03/2026, 18:01:57	sensor-03	Kestrel Basin	Central Asia	earthquake	0.89	0.2579	processing-1
31/03/2026, 18:01:56	sensor-03	Kestrel Basin	Central Asia	conventional_explosion	3.91	0.3034	processing-2
31/03/2026, 18:01:55	sensor-07	Argent Plateau	South Atlantic	earthquake	0.73	0.2407	processing-3
31/03/2026, 18:01:54	sensor-07	Argent Plateau	South Atlantic	conventional_explosion	7.31	0.2566	processing-3
31/03/2026, 18:01:53	sensor-07	Argent Plateau	South Atlantic	earthquake	0.67	0.2566	processing-1
31/03/2026, 18:01:52	sensor-07	Argent Plateau	South Atlantic	earthquake	2.85	0.2566	processing-1
31/03/2026, 18:01:52	sensor-04	Ember Fault	East Africa	earthquake	0.85	0.3129	processing-1
31/03/2026, 18:01:51	sensor-04	Ember Fault	East Africa	conventional_explosion	3.23	0.3129	processing-3

Fault Tolerance

- Health checks identify non-responsive replicas
- Broker excludes unhealthy replicas from forwarding
- Remaining replicas keep processing and writing events

> Goal: no single replica failure stops detection



Thank You

Repo:https://github.com/Fdpep/2058211_ProgMate.git